



**UNIVERSIDADE FEDERAL DO TOCANTINS
CÂMPUS UNIVERSITÁRIO DE PALMAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO**

HENRIQUE NORONHA FERNANDES

**ANÁLISE COMPARATIVA DE GERENCIAMENTO DE RECURSOS DO
SO: ABLETON LIVE VS. FL STUDIO**

PALMAS (TO)

2026

Henrique Noronha Fernandes

**Análise Comparativa de Gerenciamento de Recursos do SO: Ableton Live vs.
FL Studio**

Relatório de Sistemas Operacionais
apresentado à Universidade Federal do
Tocantins para obtenção do título de Bacharel
em Ciência da Computação. Orientador:

[Titulação] [Nome do Professor de SO]
[Sobrenome]

PALMAS (TO)

2026

SUMÁRIO

1	INTRODUÇÃO	0
2	REFERENCIAL TEÓRICO	1
2.1	Gerenciamento de Processos e <i>Threads</i>	1
2.1.1	Escalonador do Windows NT	1
2.1.2	Tempo Real Crítico vs. Não Crítico	1
2.2	Interrupções e Latência de I/O em Tempo Real	1
2.2.1	<i>Callback</i> de Áudio e Tamanho de <i>Buffer</i>	2
2.2.2	ASIO vs. WDM/DirectSound	2
2.2.3	Por que DAWs não usam Escalonamento de Prazos	2
2.3	Gerenciamento de Memória e Paginação	2
2.3.1	<i>Eager</i> vs. <i>Lazy Allocation</i>	3
2.3.2	<i>Private Bytes</i>	3
3	METODOLOGIA	4
3.1	Ambiente de Teste e Isolamento de Variáveis	4
3.2	Instrumentação e Coleta de Métricas	4
3.3	Protocolo de Teste 1: Consumo em Repouso (<i>Idle</i>)	4
3.4	Protocolo de Teste 2: Carga <i>I/O-bound</i> com 20 Trilhas AIFF .	5
3.5	Protocolo de Teste 3: Escalonamento de <i>Threads</i>	5
3.6	Protocolo de Teste 4: Latência com Driver Nativo	5
3.7	Protocolo de Teste 5: Estresse CPU-bound	5
4	RESULTADOS	6
4.1	Teste 1 — Consumo em Repouso (<i>Idle</i>)	6
4.1.1	CPU em repouso	6

4.1.2	RAM em repouso	7
4.1.3	<i>Threads</i> em repouso	7
4.2	Teste 2 — Carga <i>I/O-bound</i>: 20 Trilhas AIFF	7
4.2.1	CPU com 20 trilhas	8
4.2.2	RAM com 20 trilhas	8
4.2.3	<i>Threads</i> com 20 trilhas	9
4.3	Teste 3 — Escalonamento de <i>Threads</i> (Process Explorer)	10
4.3.1	Prioridade de escalonamento	10
4.3.2	Composição de <i>threads</i> por módulo	11
4.4	Consumo de CPU e Comportamento Multithread	12
4.5	Uso de Memória RAM	12
4.6	Análise de Latência e Buffer	12
5	DISCUSSÃO	13
5.1	Análise Comparativa — Teste 1 (Repouso)	13
5.2	Análise Comparativa — Teste 2 (20 Trilhas AIFF)	13
5.3	Análise Comparativa — Teste 3 (Escalonamento de <i>Threads</i>) .	13
6	CONCLUSÃO	15
	REFERÊNCIAS	16

1 INTRODUÇÃO

Historicamente, os Sistemas Operacionais (SO) evoluíram de simples monitores de execução sequencial para ecossistemas complexos de gerenciamento de hardware e software. Enquanto as primeiras gerações de sistemas focavam estritamente na abstração de comandos básicos e organização de arquivos, os sistemas contemporâneos precisam orquestrar uma arquitetura de múltiplos núcleos (*multicore*), memória virtual e interrupções em milissegundos. No cenário contemporâneo, os computadores pessoais atuam como Estações de Trabalho de Áudio Digital (DAWs — *Digital Audio Workstations*), executando softwares complexos que exigem do SO uma orquestração extremamente eficiente dos componentes de hardware. Nesse contexto, a performance de uma DAW não depende apenas da força bruta do processador, mas de como o sistema operacional gerencia as *threads* de execução, aloca a memória e lida com operações de entrada e saída (I/O) em alta velocidade.

O grande desafio imposto por essas aplicações é a necessidade de operar sob restrições de tempo real. Conforme a classificação de Silberschatz, Galvin e Gagne (2015), as DAWs enquadram-se como sistemas de tempo real não crítico (*soft real-time*). Nesses sistemas, embora o atraso no atendimento de um processo não cause uma falha fatal no computador, ele resulta em uma severa degradação da qualidade do serviço — nesse caso, manifestando-se como falhas e artefatos sonoros indesejados. Para evitar essa degradação, o sistema operacional precisa garantir que as *threads* de áudio tenham preferência sobre processos comuns, minimizando a latência de despacho e a latência de interrupção.

Diante dessa problemática, o presente trabalho realiza uma análise comparativa do consumo e gerenciamento de recursos do sistema operacional entre duas das principais DAWs do mercado: Ableton Live e FL Studio. Busca-se mapear como cada arquitetura de software impacta o consumo de CPU, o comportamento *multithread*, o uso de memória RAM e a latência de I/O sob as mesmas condições de estresse, demonstrando na prática os impactos das decisões de escalonamento e arquitetura do SO em ambientes de alta exigência em tempo real.

2 REFERENCIAL TEÓRICO

2.1 Gerenciamento de Processos e *Threads*

Um **processo** é um programa em execução, dotado de espaço de endereçamento próprio e dos recursos a ele atribuídos pelo SO (SILBERSCHATZ; GALVIN; GAGNE, 2015). Dentro de um processo podem coexistir múltiplas ***threads***: unidades menores de execução que compartilham o mesmo espaço de endereçamento, mas mantêm pilha e registradores próprios (TANENBAUM; BOS, 2016). Essa partilha torna a troca de contexto entre *threads* muito mais barata do que entre processos — razão pela qual motores de áudio recorrem ao *multithreading* para distribuir o processamento de blocos de amostras entre os núcleos do hardware.

2.1.1 Escalonador do Windows NT

O escalonador do Windows NT é **preemptivo baseado em prioridades**, com 32 níveis (0–31): a faixa de usuário ocupa os níveis 1–15 e a faixa de tempo real, 16–31 (STALLINGS, 2017). Os valores observados neste trabalho situam-se na faixa de usuário: *Normal* (8), *Above Normal* (10) e *High* (12–13). A cada *thread* o sistema atribui uma **prioridade base** (derivada da classe do processo) e, em tempo de execução, pode aplicar um ***priority boosting*** temporário — elevação automática para compensar *starvation* ou premiar *threads* recém-acordadas de I/O (STALLINGS, 2017). O *boost* decai gradualmente até retornar à base, o que explica a divergência entre prioridade base e dinâmica observável em ferramentas como o *Process Explorer*.

2.1.2 Tempo Real Crítico vs. Não Crítico

Silberschatz, Galvin e Gagne (2015) distinguem dois regimes de tempo real. Em sistemas de **tempo real crítico** (*hard real-time*), perder um prazo é uma falha absoluta — inaceitável em aviãoica ou em dispositivos médicos. Em sistemas de **tempo real não crítico** (*soft real-time*), a perda eventual de um prazo degrada a qualidade do serviço, mas não compromete o sistema. DAWs caem nesta segunda categoria: um *buffer* não preenchido a tempo produz *dropout* audível, mas a máquina continua estável. Basta, portanto, que as *threads* de áudio operem em prioridade superior à dos processos comuns para que o escalonador as despache primeiro sob contenção.

2.2 Interrupções e Latência de I/O em Tempo Real

Uma **interrupção de hardware** é um sinal assíncrono pelo qual um periférico avisa o processador de que requer atenção imediata (SILBERSCHATZ; GALVIN;

GAGNE, 2015). Ao recebê-lo, o processador suspende a *thread* corrente, salva seu estado e transfere o controle para a **Rotina de Serviço de Interrupção** (ISR). O intervalo entre o sinal e o início da ISR é a **latência de interrupção**; somada ao tempo de troca de contexto até a *thread* destino, resulta na **latência de despacho** (STALLINGS, 2017). Em áudio, qualquer atraso de despacho que exceda o período do *buffer* se manifesta como silêncio ou artefato sonoro.

2.2.1 *Callback* de Áudio e Tamanho de *Buffer*

Drivers de áudio operam no modelo ***pull callback***: o hardware solicita periodicamente um bloco de N amostras e o motor de áudio tem até o prazo do próximo bloco para entregá-lo (FONS, 2010). A relação entre *buffer* e latência é direta:

$$L = \frac{N}{f_s} \quad (1)$$

onde L é a latência em segundos, N o tamanho do bloco em amostras e f_s a taxa de amostragem. A 44.100 Hz, $N = 256$ corresponde a $\approx 5,8$ ms. Reduzir N diminui a latência perceptível, mas multiplica a frequência das interrupções — e, com ela, a pressão sobre o escalonador (KIM; HAHN, 2003).

2.2.2 ASIO vs. WDM/DirectSound

O Windows expõe duas rotas de áudio ao aplicativo. Pelo **WDM/DirectSound**, o fluxo passa pelo *Kernel Audio Mixer* (KMixer), responsável por misturar áudio de múltiplos aplicativos antes de enviá-lo ao hardware (Microsoft Corporation, 2023a). Cada ciclo de *buffer* acorda o KMixer, acrescentando latência e interrupções extras. Já o protocolo **ASIO** (*Audio Stream Input/Output*) entrega ao driver do fabricante acesso direto ao hardware, ignorando o KMixer (Microsoft Corporation, 2023a). O ganho é uma redução substancial de latência e de carga sobre o escalonador.

2.2.3 Por que DAWs não usam Escalonamento de Prazos

Liu e Layland (1973) formalizaram o escalonamento em tempo real com o algoritmo *Rate Monotonic*, e Buttazzo (2011) mostraram que a garantia formal de prazos exige análise de pior caso (WCET). DAWs, como sistemas *soft real-time*, dispensam esse rigor: limitam-se a elevar a prioridade das *threads* de áudio no escalonador genérico do SO, trocando garantia formal por simplicidade de implementação.

2.3 Gerenciamento de Memória e Paginação

Cada processo recebe do SO um espaço de endereçamento virtual contíguo e isolado, independente da memória física disponível (SILBERSCHATZ; GALVIN; GAGNE,

2015). A tradução entre endereço virtual e físico é feita pela MMU com base nas tabelas de páginas do *kernel*. Na **paginação por demanda**, uma página só é mapeada a um quadro físico quando acessada pela primeira vez — evento chamado de **falta de página** (*page fault*) (TANENBAUM; BOS, 2016). Resolvê-la exige suspender a *thread*, localizar (ou ler do disco) a página e retomar a execução: uma latência não determinística que pode variar de microssegundos a milissegundos.

Para áudio em tempo real, uma falta de página durante o processamento de um bloco é especialmente cara: a *thread* fica bloqueada enquanto o prazo do *buffer* avança (KIM; HAHN, 2003). Sistemas críticos fixam páginas na RAM via `mlock()` (Linux) ou `VirtualLock()` (Windows); DAWs comerciais costumam optar por alocação antecipada (*eager allocation*) como equivalente prático.

2.3.1 *Eager vs. Lazy Allocation*

Tanenbaum e Bos (2016) contrastam dois modelos de alocação:

- ***Eager allocation***: reserva e inicializa toda a memória prevista logo na inicialização do processo. Elimina faltas de página no caminho crítico, ao custo de uma pegada elevada em repouso.
- ***Lazy allocation***: aloca recursos só quando demandados. Mantém pegada mínima em repouso, mas introduz latência não determinística no momento da primeira alocação.

2.3.2 *Private Bytes*

O contador *Private Bytes* do Windows mede a memória virtual comprometida exclusivamente pelo processo — ignorando DLLs e demais regiões compartilhadas (STALLINGS, 2018). É a métrica mais fiel ao custo real de memória da aplicação, por refletir o que precisaria ser paginado caso o sistema ficasse sem RAM física. Em DAWs, o crescimento desse contador ao carregar trilhas reflete sobretudo os *buffers* de pré-leitura e as estruturas de decodificação mantidas em memória para reprodução de baixa latência.

3 METODOLOGIA

O presente estudo adota uma abordagem quantitativa e comparativa para analisar o gerenciamento de recursos e o desempenho de *Digital Audio Workstations* (DAWs) sob a ótica de Sistemas Operacionais. O foco reside na interação entre o *kernel* do Windows e os motores de áudio do **Ableton Live 12** e **FL Studio 24**.

3.1 Ambiente de Teste e Isolamento de Variáveis

Para garantir a reprodutibilidade, o ambiente foi configurado para minimizar a interferência de processos em segundo plano:

- **Camada de Abstração de Áudio:** Cada DAW foi testada com seu driver de áudio nativo, preservando as escolhas arquiteturais de cada fabricante. O FL Studio utilizou o *FL Studio ASIO*; o Ableton Live utilizou seu driver nativo padrão.
- **Configuração de saída:** O *Fruity Limiter* nativo do FL Studio foi removido do canal Master antes dos testes, garantindo paridade com a arquitetura aditiva do Ableton Live.
- **Hardware Host:** Estação de trabalho com processador multinúcleo (DESKTOP-6GDB43U), permitindo observar o escalonamento de *threads* pelo gerenciador do sistema.

3.2 Instrumentação e Coleta de Métricas

- **Performance Monitor (PerfMon):** Ferramenta nativa do Windows utilizada para capturar contadores do *kernel* a 1 Hz, exportados via `relog` para formato `.csv` (Microsoft Corporation, 2023b). Métricas coletadas:
 - % Tempo de Processador (`_Total`)
 - Bytes Particulares (`Private Bytes`) por processo
 - Contagem de Threads por processo
- **Process Explorer (Sysinternals):** Inspeção detalhada de *threads* individuais, prioridades de escalonamento e consumo de CPU por *thread*.

3.3 Protocolo de Teste 1: Consumo em Repouso (*Idle*)

DAW aberta sem projeto carregado. Coleta de 2,5 minutos a 1 Hz via PerfMon.

3.4 Protocolo de Teste 2: Carga *I/O-bound* com 20 Trilhas AIFF

Reprodução simultânea em *loop* de 20 trilhas de áudio no formato **AIFF** (PCM não comprimido), evitando ciclos de CPU destinados à descompressão. Coleta de 3 minutos a 1 Hz.

3.5 Protocolo de Teste 3: Escalonamento de *Threads*

Inspeção via *Process Explorer* (Sysinternals) com o projeto de 20 trilhas em reprodução. Registraram-se: total de *threads* ativas, prioridade base e dinâmica da *thread* principal, módulos de origem das *threads* e CPU consumida por *thread*.

3.6 Protocolo de Teste 4: Latência com Driver Nativo

3.7 Protocolo de Teste 5: Estresse CPU-bound

4 RESULTADOS

4.1 Teste 1 — Consumo em Repouso (*Idle*)

O Teste 1 estabeleceu o consumo de recursos de ambas as DAWs sem nenhum projeto carregado. Os dados foram coletados durante aproximadamente 2,5 minutos, resultando em 146 amostras válidas para o Ableton Live 12 e 145 para o FL Studio 24.

Tabela 1 – Resumo estatístico — Teste 1: consumo em repouso

Métrica	DAW	Mínimo	Média	Máximo
% CPU	Ableton Live 12	0,00%	10,95%	33,73%
	FL Studio (FL64)	0,00%	12,38%	37,72%
Private Bytes (MB)	Ableton Live 12	338,6	536,8	538,3
	FL Studio (FL64)	95,6	95,8	95,8
Threads ativas	Ableton Live 12	8	54,8	56
	FL Studio (FL64)	44	44,0	45

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.1.1 CPU em repouso

O FL Studio registrou média de 12,38%, ligeiramente superior ao Ableton Live (10,95%), com pico máximo de 37,72% frente a 33,73%. A diferença é sistemática ao longo de toda a sessão (Figura 1), evidenciando processos internos contínuos mesmo sem projeto ativo.

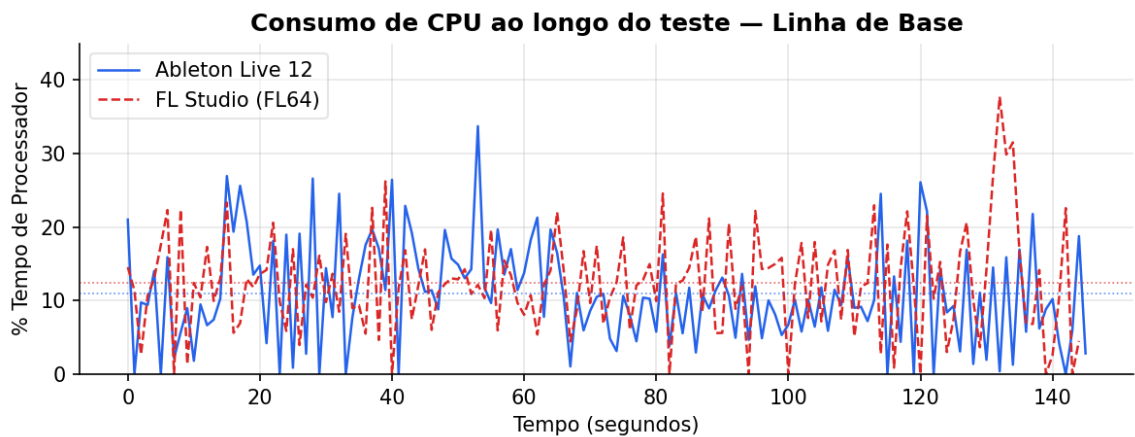


Figura 1 – Consumo de CPU em repouso — Teste 1.

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.1.2 RAM em repouso

O Ableton Live alocou em média 536,8 MB em repouso, valor $5,6\times$ superior ao FL Studio (95,8 MB), que manteve consumo praticamente constante (variação $< 0,2$ MB) (Figura 2).

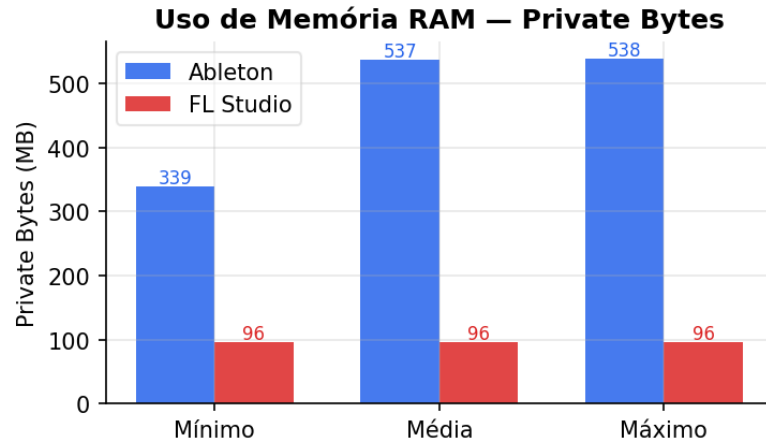


Figura 2 – Uso de RAM (*Private Bytes*) em repouso — Teste 1.

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.1.3 *Threads* em repouso

O Ableton Live manteve em média 54,8 *threads* ativas em repouso, contra 44 *threads* estáveis do FL Studio (Figura 3).

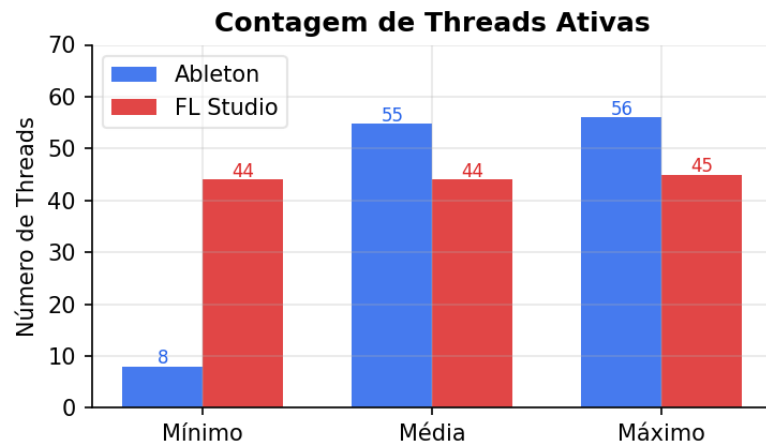


Figura 3 – Contagem de *threads* em repouso — Teste 1.

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.2 Teste 2 — Carga *I/O-bound*: 20 Trilhas AIFF

O Teste 2 introduziu carga de trabalho real com a reprodução simultânea em *loop* de 20 trilhas de áudio AIFF (PCM não comprimido) em ambas as DAWs. Foram coletadas

178 amostras para o Ableton Live 12 e 169 para o FL Studio 24, durante aproximadamente 3 minutos cada.

Tabela 2 – Resumo estatístico — Teste 2: 20 trilhas AIFF em reprodução

Métrica	DAW	Mínimo	Média	Máximo
% CPU	Ableton Live 12	0,00%	3,74%	20,11%
	FL Studio (FL64)	0,00%	4,80%	31,16%
Private Bytes (MB)	Ableton Live 12	601,6	601,7	604,5
	FL Studio (FL64)	137,6	147,6	147,6
Threads ativas	Ableton Live 12	54	55,3	56
	FL Studio (FL64)	26	27,0	28

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.2.1 CPU com 20 trilhas

Com a carga *I/O-bound* ativa, ambas as DAWs apresentaram redução expressiva no consumo médio de CPU em relação ao repouso: o Ableton Live caiu de 10,95% para 3,74% (−6,21 pp) e o FL Studio de 12,38% para 4,80% (−7,58 pp). A Figura 4 ilustra o comportamento ao longo do teste, e a Figura 5 compara os dois cenários.

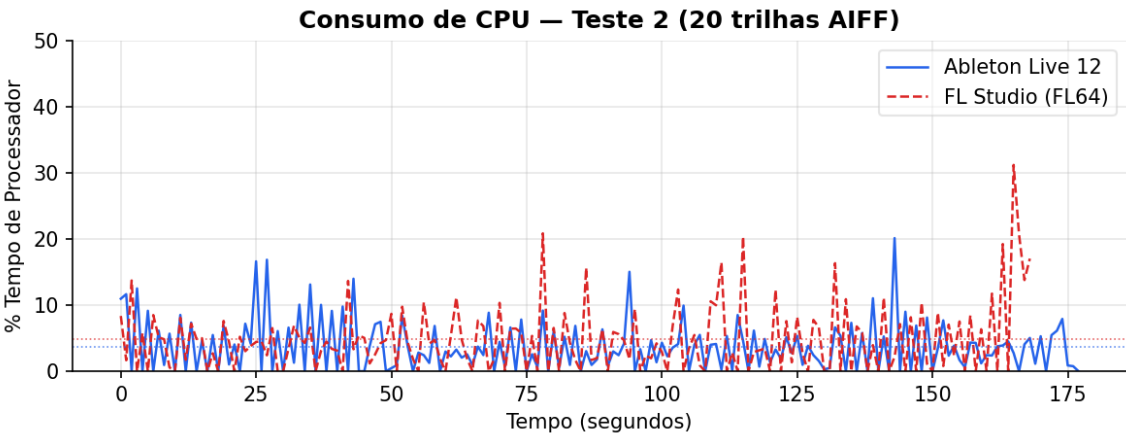


Figura 4 – Consumo de CPU durante reprodução de 20 trilhas AIFF — Teste 2.

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.2.2 RAM com 20 trilhas

O Ableton Live elevou seu consumo de 536,8 MB para 601,7 MB (+64,9 MB), mantendo-se estável após o carregamento das trilhas. O FL Studio partiu de 95,8 MB e estabilizou em 147,6 MB (+51,8 MB). A proporção entre as duas DAWs se manteve: o Ableton consumiu 4,1× mais RAM que o FL Studio com 20 trilhas ativas (Figura 6).

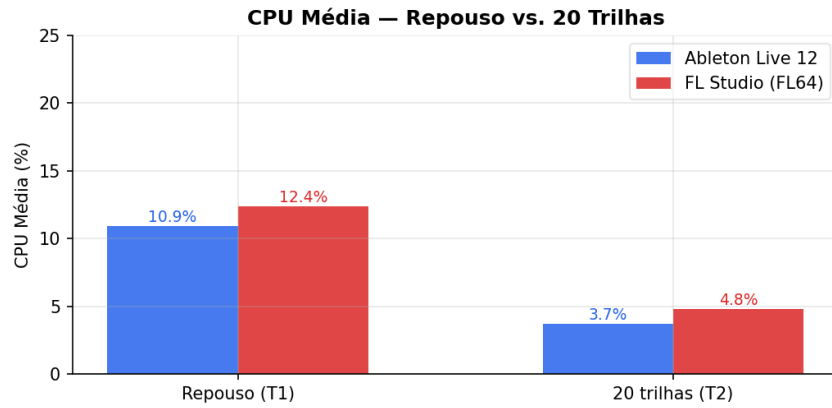


Figura 5 – CPU média comparativa: repouso (Teste 1) vs. 20 trilhas (Teste 2).

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

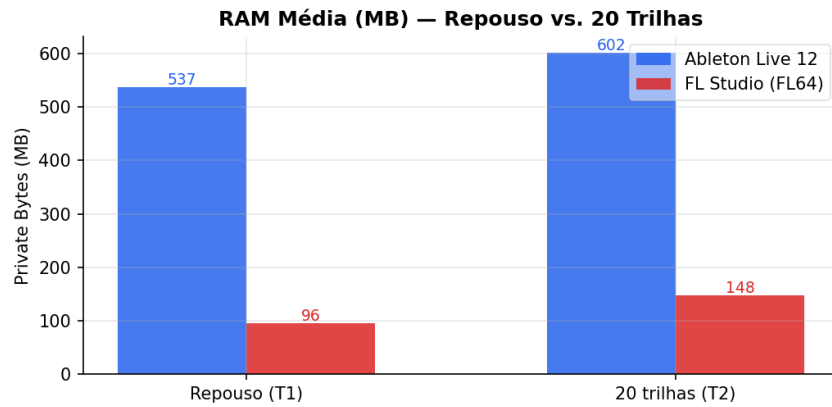


Figura 6 – RAM média comparativa: repouso (Teste 1) vs. 20 trilhas (Teste 2).

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.2.3 *Threads* com 20 trilhas

O resultado mais surpreendente do Teste 2 foi a queda abrupta de *threads* do FL Studio: de 44 em repouso para 27 com o projeto ativo — redução de 38%. O Ableton Live, por sua vez, manteve sua contagem praticamente estável (54,8 → 55,3), confirmando a arquitetura de *threads* permanentes (Figura 7).

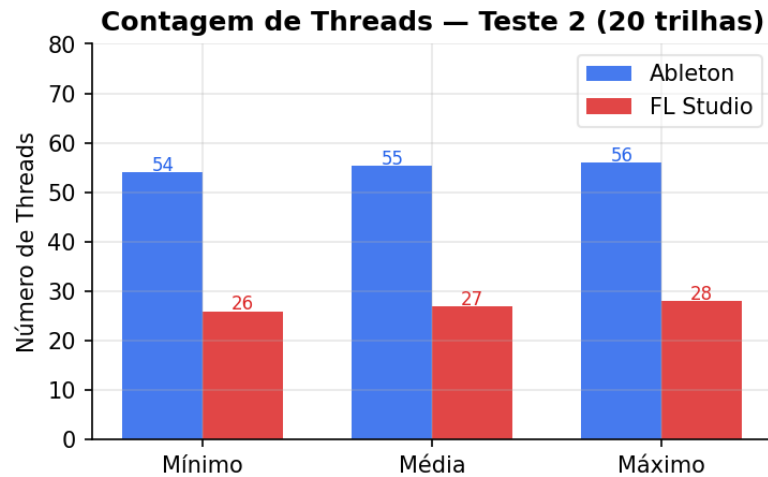


Figura 7 – Contagem de *threads* com 20 trilhas ativas — Teste 2.

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.3 Teste 3 — Escalonamento de *Threads* (Process Explorer)

O Teste 3 utilizou o *Process Explorer* da Sysinternals para inspecionar a composição interna de *threads* de cada DAW com o projeto de 20 trilhas AIFF em reprodução ativa. A Tabela 3 resume os dados extraídos da aba *Threads* de cada processo.

Tabela 3 – Escalonamento de *threads* — Teste 3 (Process Explorer)

Parâmetro	FL Studio (FL64)	Ableton Live 12
Total de <i>threads</i>	29	55
Prioridade base (thread principal)	10 (Above Normal)	8 (Normal)
Prioridade dinâmica	12 (High)	10 (Above Normal)
CPU da thread principal	0,93%	0,75%
Thread <i>Time Critical</i> (15)?	Não	Não
Módulo dominante	QuickFontCache (UI)	ucrtbase (C runtime)
Thread de driver de áudio	ilwasapi2asio_x64	DSOUND.dll

Fonte: dados coletados pelos autores via Process Explorer, 26 maio 2026.

4.3.1 Prioridade de escalonamento

O achado mais relevante do Teste 3 foi a diferença nas prioridades de escalonamento da *thread* principal de cada processo (Figura 8). O FL Studio agenda sua *thread* principal com prioridade base 10 (*Above Normal*) e prioridade dinâmica 12 (*High*), enquanto o Ableton Live utiliza prioridade base 8 (*Normal*) e dinâmica 10. Nenhuma das duas DAWs atingiu o nível *Time Critical* (prioridade 15), que caracterizaria sistemas de tempo real crítico.

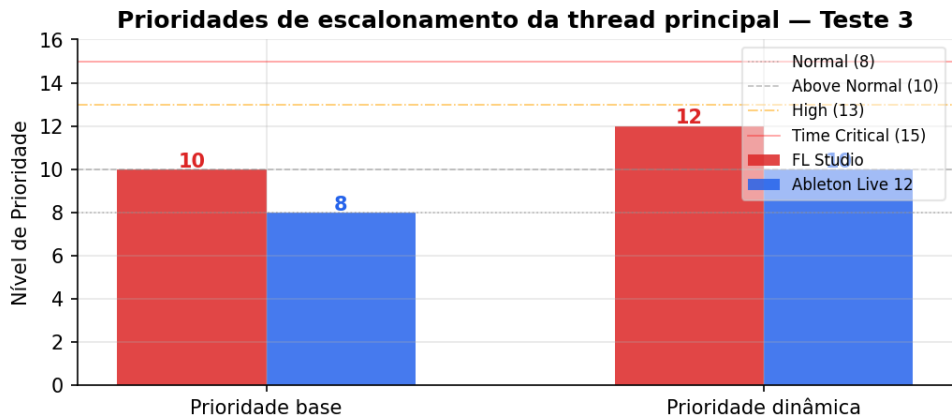


Figura 8 – Prioridades de escalonamento da *thread* principal — Teste 3. As linhas de referência indicam os níveis definidos pelo escalonador do Windows NT.

Fonte: dados coletados pelos autores via Process Explorer, 26 maio 2026.

4.3.2 Composição de *threads* por módulo

A Figura 9 detalha a distribuição de *threads* por módulo em cada processo. No FL Studio, 14 das 29 *threads* pertencem ao módulo QuickFontCache — responsável pelo sistema de renderização de fontes da interface gráfica — enquanto apenas 5 *threads* estão associadas ao FLEngine_x64, o motor de áudio propriamente dito. Uma *thread* dedicada ao driver ilwasapi2asio_x64 confirma o uso do protocolo ASIO nativo do FL Studio.

No Ableton Live, aproximadamente 30 das 55 *threads* provêm do módulo ucrtbase.dll (Universal C Runtime), indicando uso intensivo de *threads* de suporte da biblioteca padrão do C. O driver de áudio é representado por *threads* do DSOUND.dll (DirectSound), e uma *thread* do WINMM.dll gerencia dispositivos MIDI e joystick.

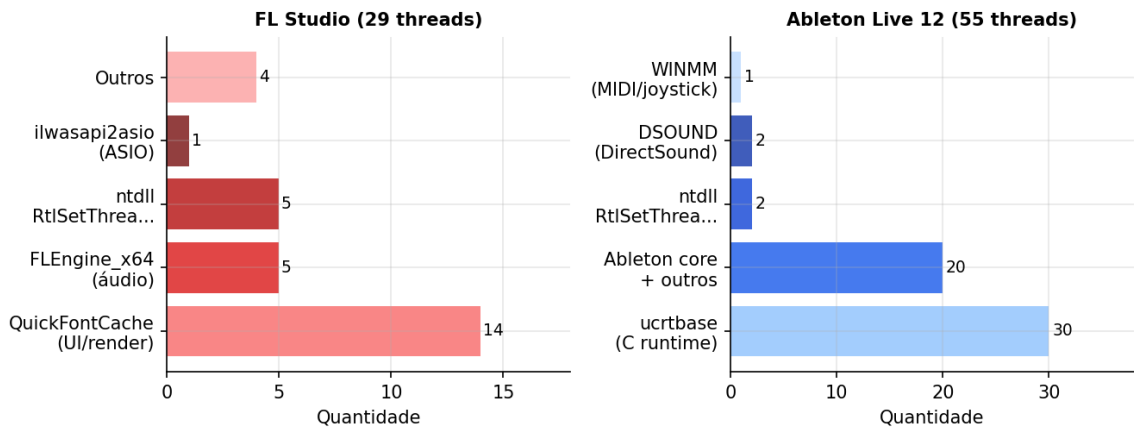


Figura 9 – Composição de *threads* por módulo — Teste 3.

Fonte: dados coletados pelos autores via Process Explorer, 26 maio 2026.

4.4 Consumo de CPU e Comportamento Multithread

4.5 Uso de Memória RAM

4.6 Análise de Latência e Buffer

5 DISCUSSÃO

5.1 Análise Comparativa — Teste 1 (Repouso)

Os resultados do Teste 1 revelam perfis de inicialização distintos. O **Ableton Live** adota *eager allocation*: aloca memória e cria *threads* antecipadamente (≈ 537 MB, 55 *threads*), garantindo baixa latência de despacho desde a inicialização (SILBERSCHATZ; GALVIN; GAGNE, 2015). O **FL Studio** demonstrou *lazy allocation*: mantém apenas estruturas mínimas em repouso (≈ 96 MB, 44 *threads*), postergando alocações para o carregamento do projeto (TANENBAUM; BOS, 2016).

5.2 Análise Comparativa — Teste 2 (20 Trilhas AIFF)

O Teste 2 revelou três achados relevantes.

Primeiro, a **redução de CPU sob carga *I/O-bound*** em ambas as DAWs — contraintuitiva à primeira vista — é explicada pelo comportamento do driver de áudio nativo: ao assumir o controle exclusivo do hardware de saída, o motor de áudio elimina a sobrecarga do *Kernel Mixer* do Windows, reduzindo interrupções desnecessárias ao escalonador.

Segundo, o **crescimento proporcional de RAM** foi semelhante em termos absolutos (+65 MB no Ableton, +52 MB no FL Studio), sugerindo que o custo de memória por trilha AIFF é comparável entre as arquiteturas. A diferença total (601 MB vs. 148 MB) reflete majoritariamente a pré-alocação do Ableton em repouso, e não o custo incremental das trilhas.

Terceiro, a **queda de *threads* no FL Studio** (de 44 para 27) ao carregar o projeto indica que o motor de áudio do FL Studio consolida algumas *threads* de gerenciamento geral em *threads* especializadas de áudio, reorganizando o perfil de paralelismo conforme a carga (STALLINGS, 2017).

5.3 Análise Comparativa — Teste 3 (Escalação de *Threads*)

O Teste 3 revelou que as duas DAWs adotam estratégias de escalonamento distintas no nível do *kernel* do Windows.

O **FL Studio** opera com prioridade de *thread* mais elevada (base 10, dinâmica 12), o que significa que o escalonador do Windows concede ao processo maior fatia de tempo de CPU em situações de concorrência com outros processos do sistema. Conforme Silberschatz, Galvin e Gagne (2015), processos com prioridade mais alta têm preferência no despacho em escalonadores preemptivos baseados em prioridade, como o do Windows NT. Contudo, o maior número de *threads* do tipo `QuickFontCache` sugere que parte do

orçamento de CPU do processo é consumido pela camada de renderização da interface gráfica, e não exclusivamente pelo motor de áudio.

O **Ableton Live**, embora opere em prioridade base inferior (8), distribui sua carga em 55 *threads*. A predominância de *threads ucrtbase* indica uso de um modelo de concorrência baseado em *thread pool* da biblioteca padrão C, o que permite ao sistema operacional escalonar dinamicamente a carga entre núcleos disponíveis (TANENBAUM; BOS, 2016). Essa estratégia favorece a escalabilidade em processadores com muitos núcleos, ainda que a prioridade individual de cada *thread* seja menor.

O fato de nenhuma das DAWs utilizar prioridade *Time Critical* (15) confirma o enquadramento de ambas como sistemas *soft real-time*: o sistema operacional pode interrompê-las em favor de processos de sistema críticos, sem que isso cause falha fatal — apenas eventual degradação perceptível no áudio (STALLINGS, 2017).

6 CONCLUSÃO

REFERÊNCIAS

BUTTAZZO, G. C. **Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications**. 3. ed. New York: Springer, 2011.

FONS, J. Real-time audio processing on general purpose operating systems. In: **Linux Audio Conference**. Utrecht: [s.n.], 2010.

KIM, Y.; HAHN, M. Improving real-time audio performance under general purpose operating systems. **IEICE Transactions on Information and Systems**, E86-D, n. 1, p. 192–196, 2003.

LIU, C. L.; LAYLAND, J. W. Scheduling algorithms for multiprogramming in a hard-real-time environment. **Journal of the ACM**, v. 20, n. 1, p. 46–61, 1973.

Microsoft Corporation. **Audio Drivers for Windows: ASIO Architecture**. [S.l.], 2023. Disponível em: <<https://docs.microsoft.com/en-us/windows-hardware/drivers/audio>>. Acesso em: 26 maio 2026.

Microsoft Corporation. **Windows Performance Monitor: Technical Reference**. [S.l.], 2023. Disponível em: <<https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/perfmon>>. Acesso em: 26 maio 2026.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Fundamentos de Sistemas Operacionais**. 9. ed. Rio de Janeiro: LTC, 2015. Tradução de: Operating System Concepts, 9th ed.

STALLINGS, W. **Operating Systems: Internals and Design Principles**. 9. ed. Hoboken: Pearson, 2017.

STALLINGS, W. **Arquitetura e Organização de Computadores**. 10. ed. São Paulo: Pearson Education do Brasil, 2018. Tradução de: Computer Organization and Architecture, 10th ed.

TANENBAUM, A. S.; BOS, H. **Sistemas Operacionais Modernos**. 4. ed. São Paulo: Pearson Education do Brasil, 2016. Tradução de: Modern Operating Systems, 4th ed.